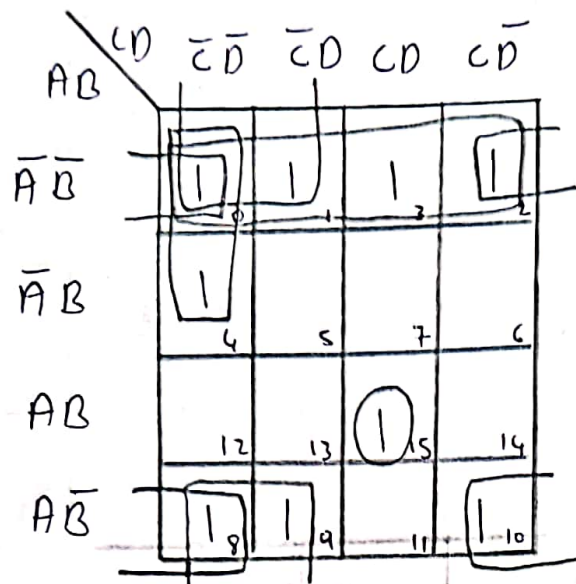
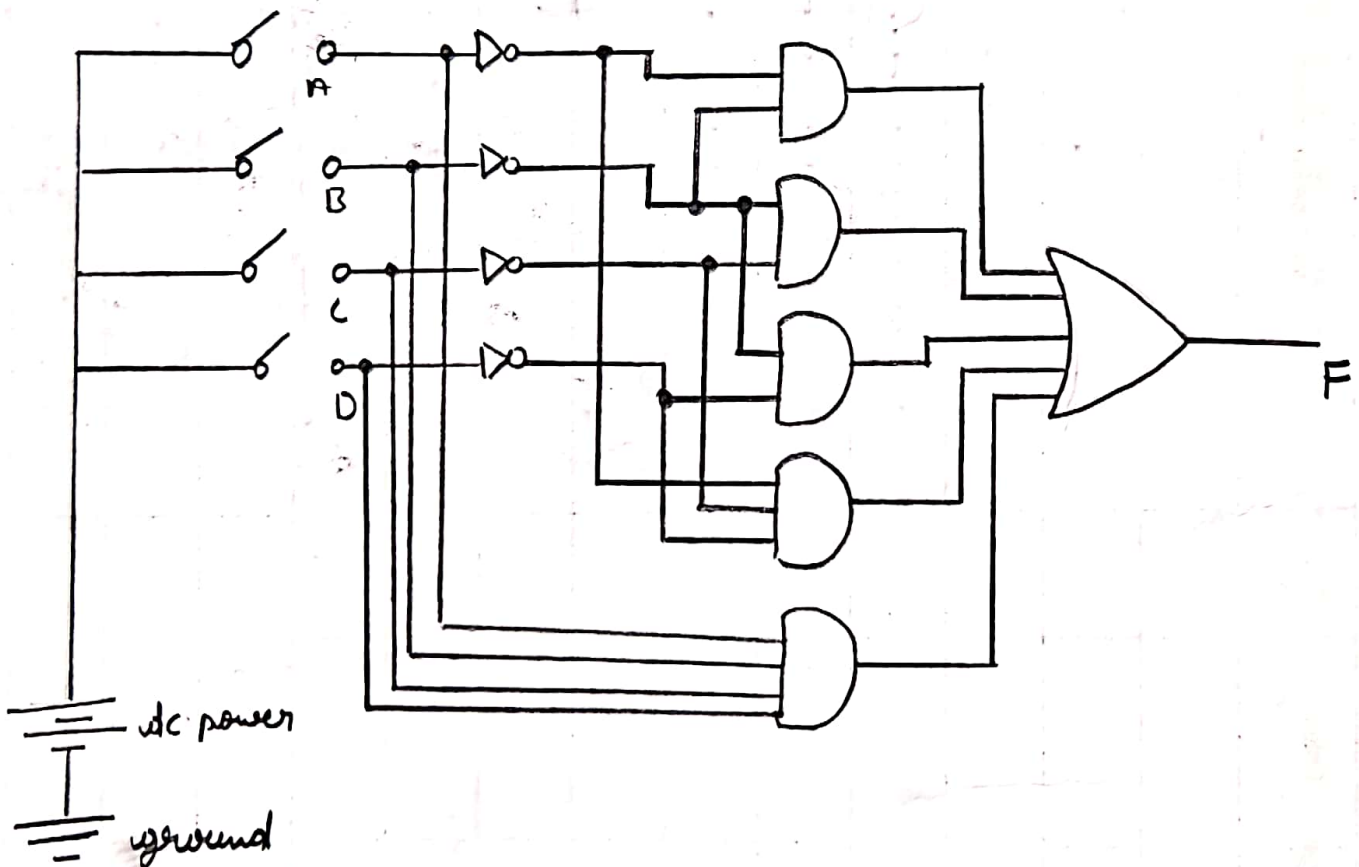


K-Map



$$F = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{B}\bar{D} + \bar{A}\bar{C}\bar{D} + ABCD$$



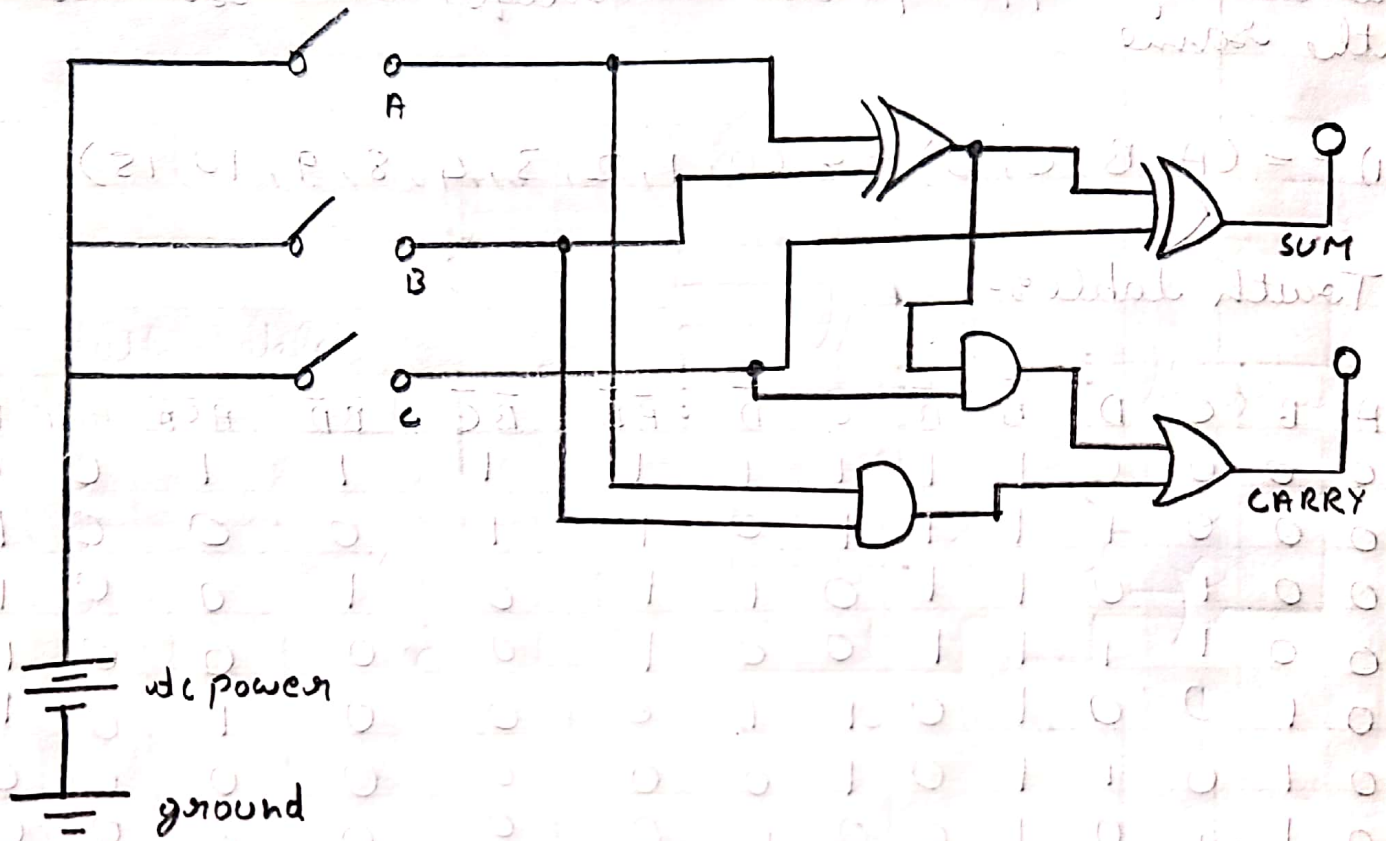
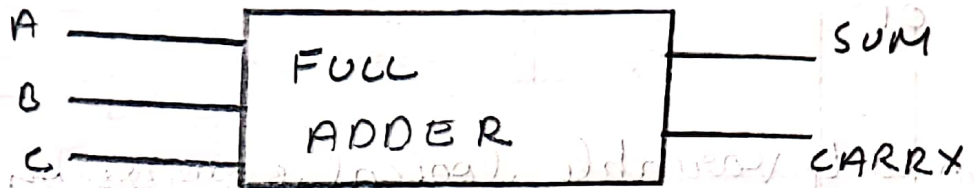
Given a 4 variable logical expression simplify it using appropriate technique and simulate the same

$$1) F = (A, B, C, D) = \Sigma (0, 1, 2, 3, 4, 8, 9, 10, 15)$$

Truth table:-

A	B	C	D	$\bar{A}$	$\bar{B}$	$\bar{C}$	$\bar{D}$	$\bar{A}\bar{B}$	$\bar{B}\bar{C}$	$\bar{B}\bar{D}$	$\bar{A}\bar{C}\bar{D}$	$ABCD$	F
0	0	0	0	1	1	1	1	1	1	1	1	0	1
0	0	0	1	1	1	1	0	1	1	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	0	0	1
0	0	1	1	1	1	0	0	1	0	0	0	0	1
0	1	0	0	1	0	1	1	0	0	0	1	0	1
0	1	0	1	1	0	1	0	0	0	0	0	0	0
0	1	1	0	1	0	0	1	0	0	0	0	0	0
0	1	1	1	1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1	0	1	1	0	0	1
1	0	0	1	0	1	1	0	0	1	0	0	0	1
1	0	1	0	0	1	0	1	0	0	1	0	0	1
1	0	1	1	0	1	0	0	0	0	0	0	0	0
1	1	0	0	0	0	1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	1	1

10



Design a 4 bit full adder and subtractor and simulate the same using basic gates.

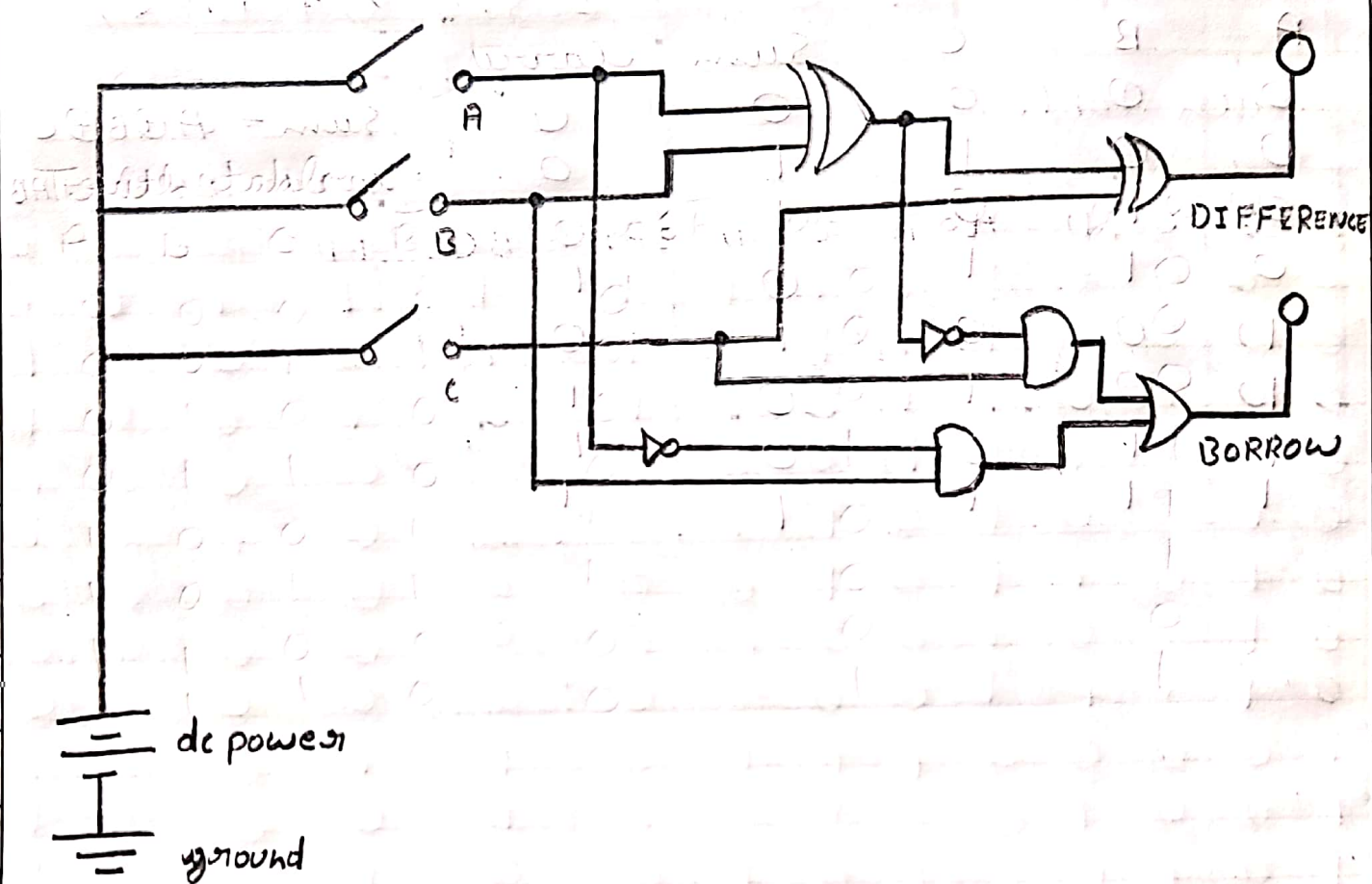
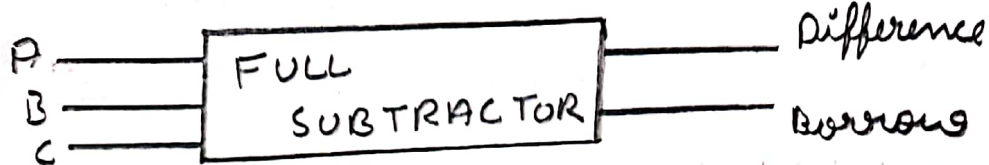
Full adder

Truth table:-

A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\text{Sum} = A \oplus B \oplus C$$

$$\text{Carry} = (A \oplus B)C + AB$$



Full subtractor

Truth table

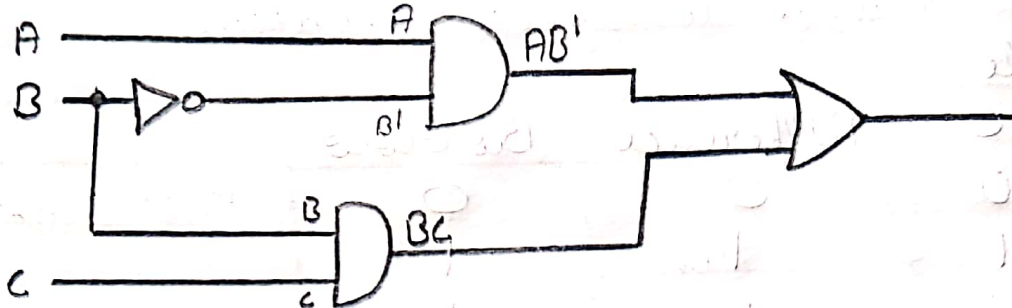
A	B	C	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	0
1	1	1	0	1

$$\text{Difference} = A \oplus B \oplus C$$

$$\text{Borrow} = \bar{A}B + C(A \oplus B)$$

Output =

Circuit diagram



Truth table

A	B	C	$\bar{B}$	$w_1 = a \bar{b}$	$w_2 = b \bar{c}$	$y = w_1 / w_2$
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	0	1	1	1	0	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

Design a verilog HDL to implement simple circuit using structural, data flow and behavioural model.

$$1) F = AB' + BC$$

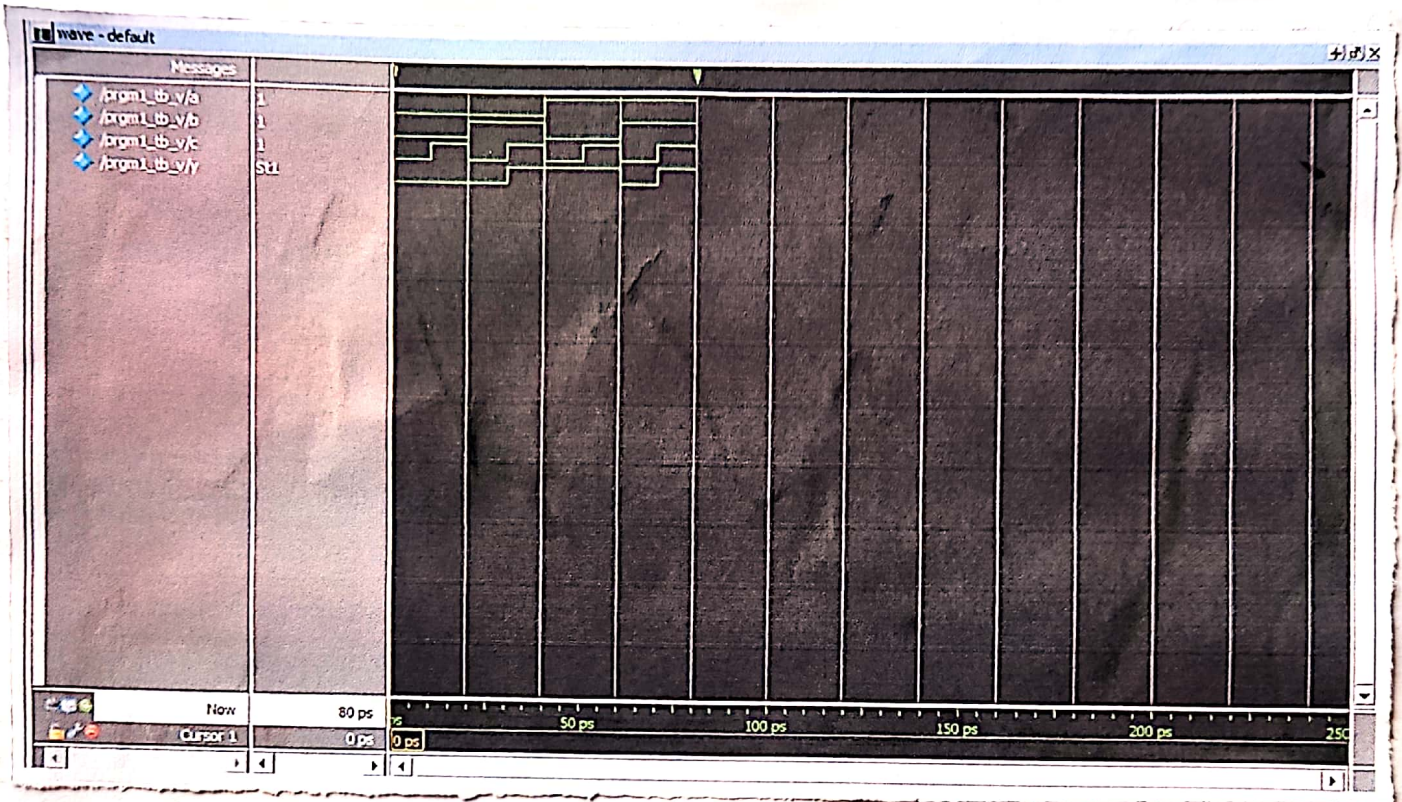
Data Flow

```
module data_flow(a, b, c, y);  
  input a, b, c;  
  output y;  
  wire w1, w2;  
  assign w1 = a & (~b);  
  assign w2 = b & c;  
  assign y = w1 | w2;  
end module
```

Structural

```
module structural(a, b, c, y);  
  input a, b, c;  
  output y;  
  NOT g1(v1, b);  
  AND g2(v2, a, v1);  
  AND g3(v3, b, c);  
  OR g4(y, v2, v3);  
end module
```

Output :-



Behavioural

```
module behavioural (
    input a, b, c;
    output y;
    reg y, v1, v2, v3;
    always @ (a, b, c)
    begin
        v1 = ~b;
        v2 = a & v1;
        v3 = b & c;
        y = v2 | v3;
    end
end module
```

Test bench

```
module tb;
    reg a;
    reg b;
    reg c;
    wire y;
    behaviour uut (.a(a), .b(b), .c(c), .y(y));
    initial begin
        a = 0; b = 0; c = 0; # 10
        a = 0; b = 0; c = 1; # 10
        a = 0; b = 1; c = 0; # 10
        a = 0; b = 1; c = 1; # 10
        a = 1; b = 0; c = 0; # 10
        a = 1; b = 0; c = 1; # 10
    end
end module
```


DATE

EXPT. TITLE :

EXP. NO. 03

PAGE NO. 06

 $a = 1 ; b = 1 ; c = 0 ; \# 10$ $a = 1 ; b = 1 ; c = 1 ; \# 10$

\$ finish;

end

end module

Module 1: Introduction to the course

Module 2: Basic concepts

Module 3: Advanced topics

Module 4: Case studies

Module 5: Summary

Module 6: Conclusion

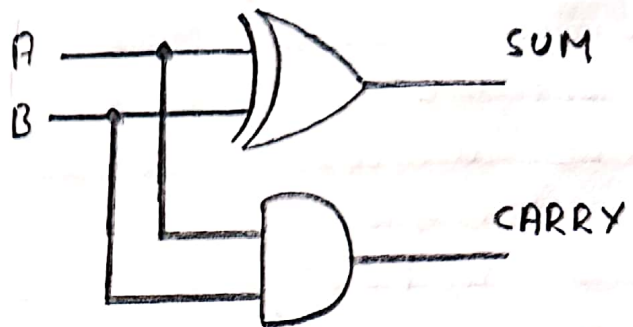
Module 7: References

- 81 pages

Truth table:-

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Circuit diagram:-



Output:-

Objects		wave - default	
Name	Value	Messages	
a	1	half_adder_tb_v/a	No Dat
b	1	half_adder_tb_v/b	No Dat
sum	S10	half_adder_tb_v/sum	No Dat
carry	S11	half_adder_tb_v/c...	No Dat

Half adder

Design a verilog HDL to implement binary adder subtractor - Half and full adder, half and full subtractor

a) Half adder

```
Module half_adder(a, b, sum, carry);  
  Input a, b;  
  output sum, carry;  
  xorl sum, a, b;  
  and(carry, a, b);  
endmodule
```

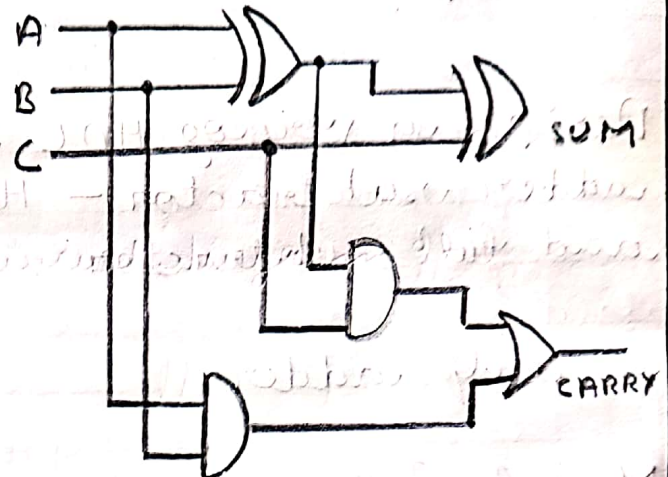
Test bench code

```
Module half_adder_tb.v;  
  reg a, b;  
  wire sum, carry;  
  half_adder uut(a, b, sum, carry);  
  initial begin  
    a = 0; b = 0; # 10  
    a = 0; b = 1; # 10  
    a = 1; b = 0; # 10  
    a = 1; b = 1; # 10  
    $finish();  
  end  
endmodule
```


Truth table :-

A	B	C	Sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit diagram :-



Output :-

Output :-

Messages		
➡ /full_adder_tb/a	1	
➡ /full_adder_tb/b	1	
➡ /full_adder_tb/cin	1	
➡ /full_adder_tb/sum	St1	
➡ /full_adder_tb/carry	St1	

Full adder

b) Full adder

Module full-adder

Input a, b, c,

output sum, carry;

Wire w1, w2, w3, w4;

xor(w1, a, b);

xor(sum, w1, c);

and(w2, a, b);

and(w3, b, c);

and(w4, c, a);

or(carry, w2, w3, w4);

endmodule

Test bench code

Module full-adder-tb;

reg a, b, c;

wire sum, carry;

full-adder uut(a, b, c, sum, carry);

initial begin

a=0; b=0; c=0; #10

a=0; b=0; c=1; #10

a=0; b=1; c=0; #10

a=0; b=1; c=1; #10

a=1; b=0; c=0; #10

a=1; b=0; c=1; #10

a=1; b=1; c=0; #10

a=1; b=1; c=1; #10

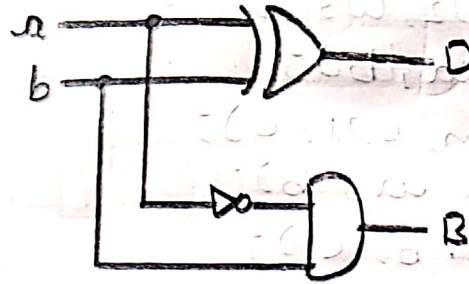
\$finish;

Truth table:-

A	B	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Output:-

Circuit diagram:-



Messages	
/b_inp/a	1
/b_inp/b	1
/b_inp/D	0
/b_inp/B	0

Half subtractor

end

endmodule

c) Half subtractor

Module half_subtractor(input a, b, output D, B):

assign D = a ^ b;

assign B = ~a & b;

endmodule

Test bench code

Module tb_top:

reg a, b;

wire D, B;

half_subtractor hs(a, b, D, B);

initial begin

\$monitor("At time %0t : a = %b, b = %b, difference = %b, borrow = %b", \$time, a, b, D, B);

a = 0; b = 0; #1;

a = 0; b = 1; #1;

a = 1; b = 0; #1;

a = 1; b = 1; #1;

\$finish();

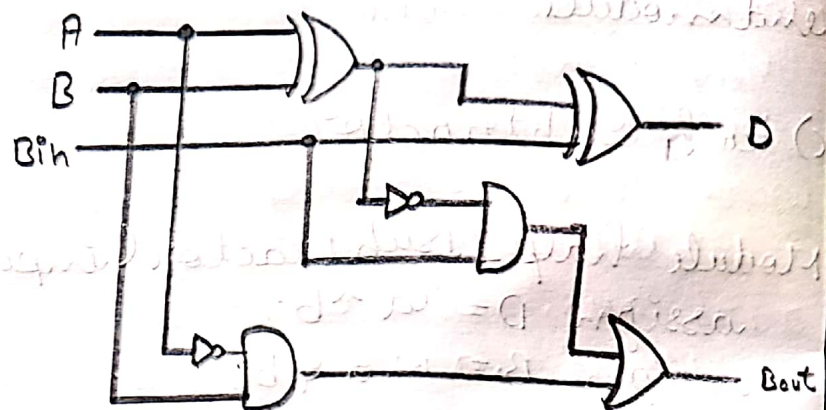
end

endmodule

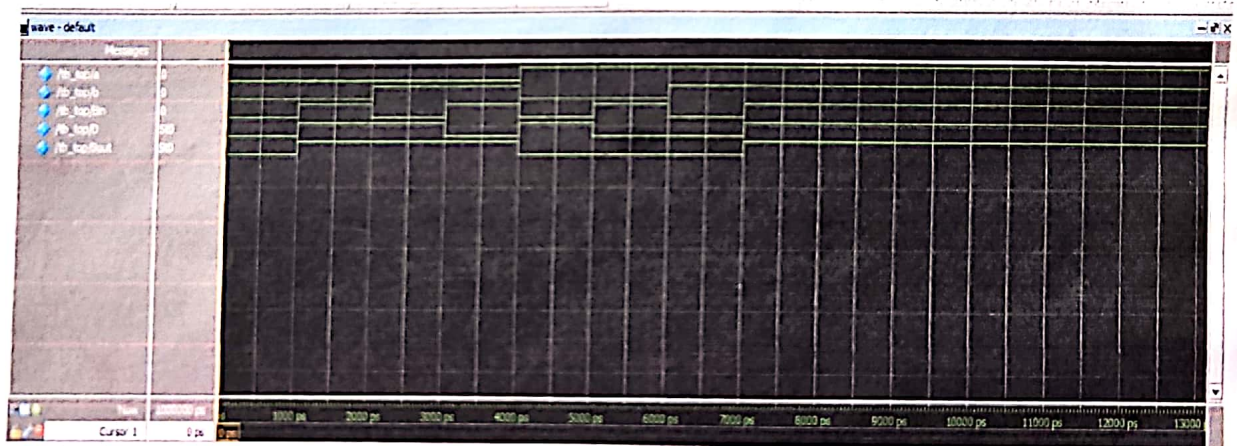
Truth table :-

a	b	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Circuit diagram :-



Output :-



Full subtractor

d) Full Subtractor

Module full_subtractor (input a, b, Bin, output, D, Bout)

assign D = $a \oplus b \oplus Bin$;

assign Bout = $(\sim a \& b) \mid (\sim (a \oplus b) \& Bin)$;

endmodule

Test bench code

Module tb_top;

reg a, b, Bin;

wire D, Bout;

full_subtractor fs(a, b, Bin, D, Bout);

initial begin

\$monitor("At time %0t: a = %b, b = %b, Bin = %b, difference = %b, borrow = %b", \$time, a, b, Bin, D, Bout);

a = 0; b = 0; Bin = 0; #1

a = 0; b = 0; Bin = 1; #1

a = 0; b = 1; Bin = 0; #1

a = 0; b = 1; Bin = 1; #1

a = 1; b = 0; Bin = 0; #1

a = 1; b = 0; Bin = 1; #1

a = 1; b = 1; Bin = 0; #1

a = 1; b = 1; Bin = 1; #1

\$finish;

end

endmodule

Subtractor

Output:-

Messages									
testbench/A	0100	0101		1000			0100		
testbench/B	0011	1011					0011		
testbench/Cin	0								
testbench/Sum	0111	0000		0011			0111		
testbench/Difference	0001	1010		1101			0001		
testbench/Cout	0								
testbench/Borrow	0								

Decimal adder

Change

Design verilog HDL to implement decimal adder

Adder

module full_adder_4bit

(

input [3:0] A,

input [3:0] B,

input cin;

output [3:0] sum,

output cout

);

assign {cout, sum} = A + B + cin;

end module

Test bench code

module testbench;

reg [3:0] A, B;

reg cin;

wire [3:0] sum;

wire cout;

Full_adder_4bit uut_adder(

.A(A),

.B(B),

.cin(cin),

.sum(sum),

.cout(cout)

);

initial begin

A = 4'b0101;

B = 4'b1011;

Cin = 0;

\$display("Full adder test:");

\$display("A=%b, B=%b, Cin=%b", A, B, Cin);

#10;

\$display("Sum=%b, cout=%d", Sum, cout);

#100;

A = 4'b1000;

B = 4'b1011;

Cin = 0;

\$display("Full adder test:");

\$display("A=%b, B=%b, Cin=%b", A, B, Cin);

#10;

\$display("Sum=%b, cout=%d", Sum, cout);

#100;

A = 4'b0100;

B = 4'b0011;

Cin = 0;

\$display("Full adder test:");

\$display("A=%b, B=%b, Cin=%b", A, B, Cin);

#10;

\$display("Sum=%b, cout=%d", Sum, cout);

#100;

\$finish;

end

endmodule

Subtractor

module Subtractor: 4bit

is

input [3:0] A,

input [3:0] B,

output [3:0] difference,

output borrow

);

assign {borrow, difference} = A - B;

endmodule

Testbench code

module testbench;

reg [3:0] A, B;

~~reg [3:0] A, B;~~

wire [3:0] ~~xxxx~~ difference;

wire borrow;

subtractor_4bit uut subtractor(

• A(A),

• B(B),

• difference(difference),

• borrow(borrow)

);

initial begin

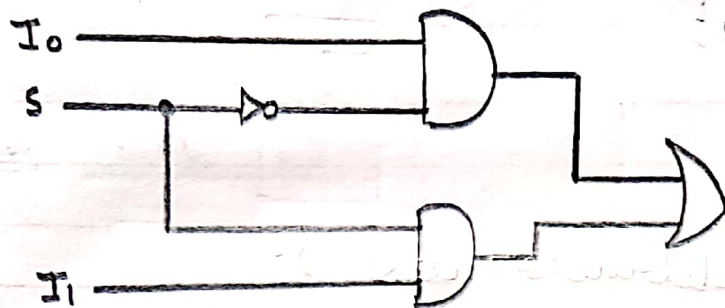
A = 4'b 0101;

B = 4'b 1011;

\$display("Subtractor test");


```
$display("A = %b, B = %b", A, B);  
#10;  
$display("Difference = %b, Borrow = %b",  
difference, borrow);  
#100;  
A = 4'b 1000;  
B = 4'b 1011;  
$display("Subtractor test");  
$display("A = %b, B = %b", A, B);  
#10;  
$display("Difference = %b, Borrow = %b",  
difference, borrow);  
#100;  
A = 4'b 0100;  
B = 4'b 0011;  
$display("Subtractor test");  
$display("A = %b, B = %b", A, B);  
#10;  
$display("Difference = %b, Borrow = %b",  
difference, borrow);  
#100;  
$finish;  
end  
endmodule
```

Circuit diagram:-



Truth table:-

S	Y
0	I_0
1	I_1

Output:-

Messages																	
⚡ /top/out	Std																
⚡ /top/d0	0																
⚡ /top/d1	0																
⚡ /top/s	1																

2:1 multiplexer

Design verilog programs to implement different types of multiplexer like 2:1, 4:1 and 8:1

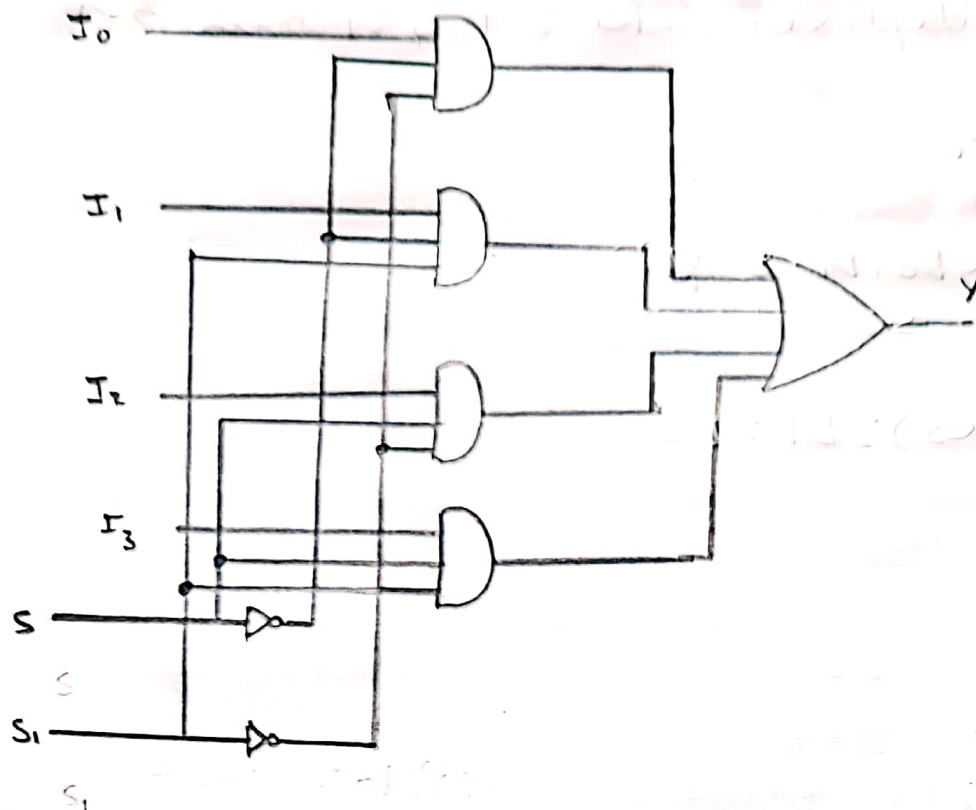
2:1 Multiplexer

```
module m21(D0, D1, S, y);  
    input D0, D1, S;  
    output y;  
    assign y = (S) ? D1 : D0;  
endmodule
```

Testbench

```
module top;  
    wire out;  
    reg d0, d1, s;  
    m21 name1(.y(out), .D0(d0), .D1(d1), .S(s));  
    initial begin  
        d0 = 1'b0;  
        d1 = 1'b0;  
        s = 1'b0;  
        #100 $finish;  
    end  
    always #10 d0 = ~d0;  
    always #20 d1 = ~d1;  
    always #10 s = ~s;  
    always @(d0 or d1 or s)  
        $monitor("At time = %t, output = %d", $time, out);  
end
```

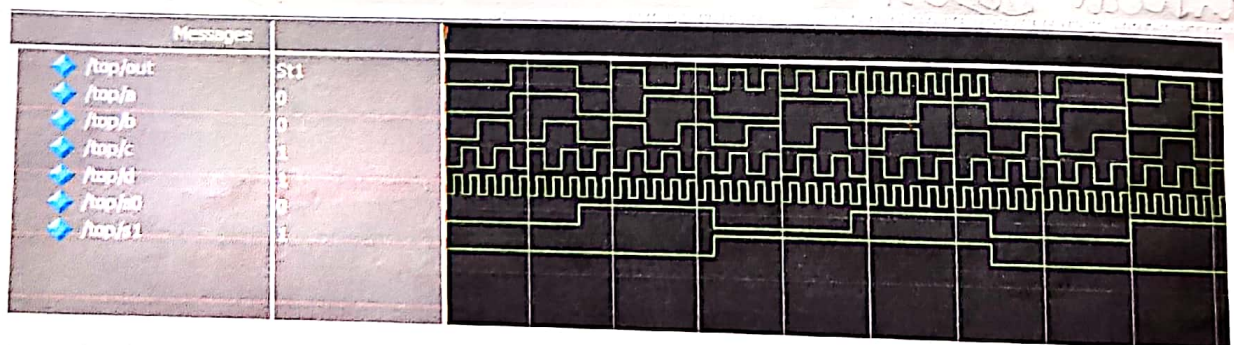
Circuit Diagram:



Truth table:

S ₁	S ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃

Output:



4:1 multiplexer


```
endmodule
```

4:1 Multiplexer

```
module m41(
```

```
    input a,
```

```
    input b, // 1'b0, 1'b1, 1'b0, 1'b1
```

```
    input c,
```

```
    input d, input s0, s1,
```

```
    output out // 1'b0, 1'b1, 1'b0, 1'b1
```

```
);
```

```
    assign out = s1 ? (s0 ? d : c) : (s0 ? b : a);
```

```
endmodule
```

Test bench code

```
module top;
```

```
    wire out;
```

```
    reg a;
```

```
    reg b;
```

```
    reg c;
```

```
    reg d;
```

```
    reg s0, s1;
```

```
    m41 mame(.out(out), .a(a), .b(b), .c(c), .d(d), .s0(s0), .s1(s1));
```

```
    initial begin
```

```
        a = 1'b0;
```

```
        b = 1'b0;
```

```
        c = 1'b0;
```

Truth table:-

S_2	S_1	S_0	χ
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7


```
d = 1'b0;
```

```
∴ s0 = 1'b0; s1 = 1'b0;
```

```
#500 $finish;
```

```
end
```

```
always #40 a = ~a;
```

```
always #20 b = ~b;
```

```
always #10 c = ~c;
```

```
always #5 d = ~d;
```

```
always #80 s0 = ~s0;
```

```
always #160 s1 = ~s1;
```

```
always @(a or b or c or d or s0 or s1)
```

```
$monitor("At time = %t, output = %d", $time,
```

```
out);
```

```
endmodule
```

8:1 Multiplexer

```
module and_gate(output a, input b, c, d, e);
```

```
assign a = b & c & d & e;
```

```
endmodule
```

```
module not_gate(output f, input g);
```

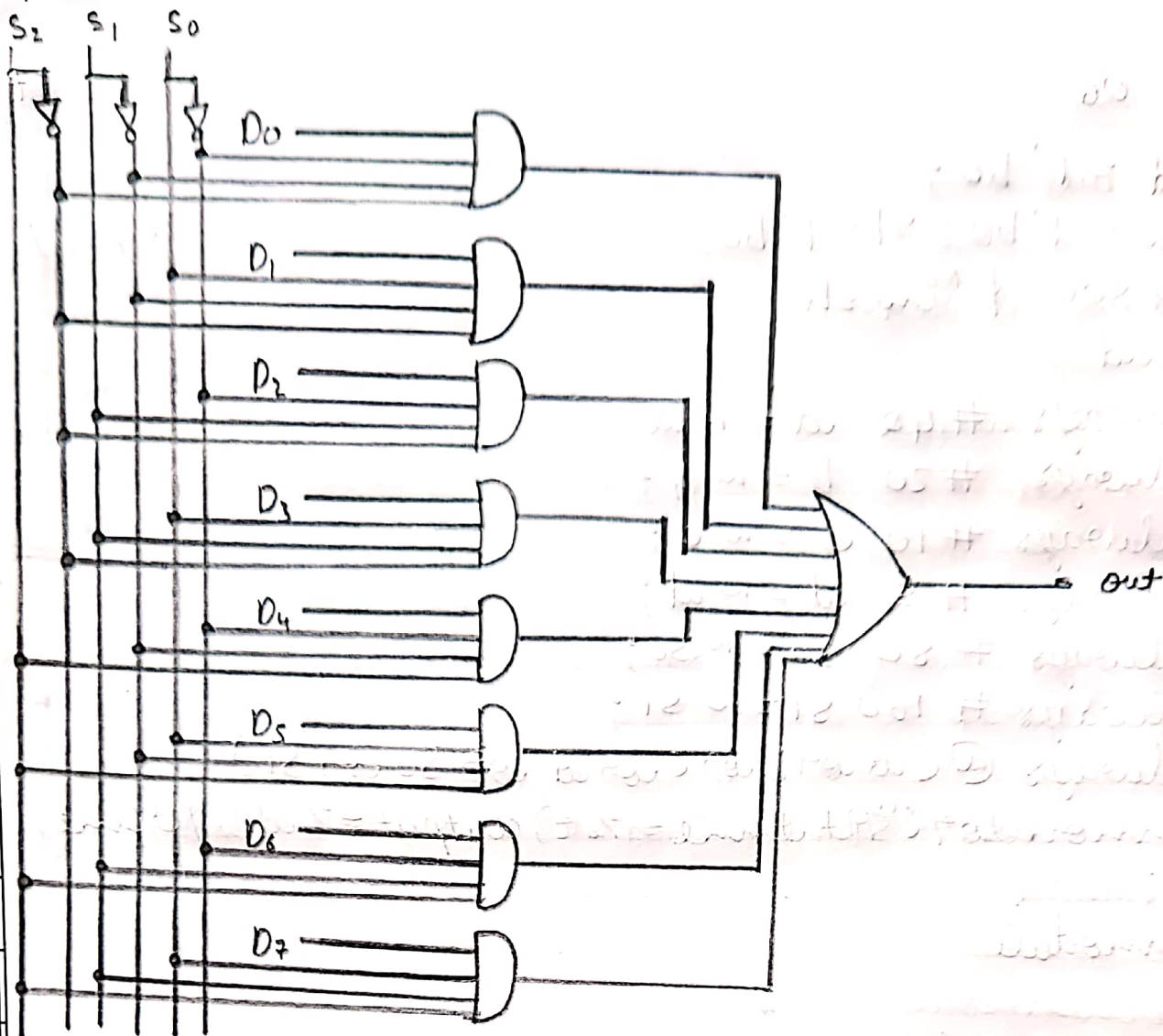
```
assign f = ~g;
```

```
endmodule
```

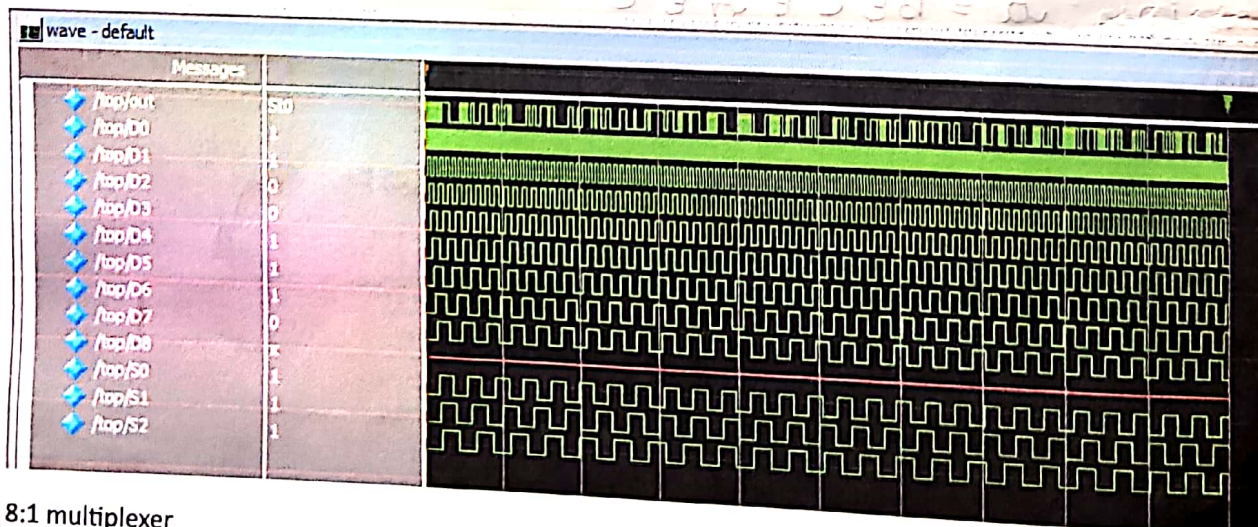
```
module or_gate(output l, input m, n, o, p, q, r, s, t);
```

```
assign l = m | n | o | p | q | r | s | t;
```

```
endmodule
```



Output :-



8:1 multiplexer


```

module m81 (out, D0, D1, D2, D3, D4, D5, D6, D7, S0, S1, S2);
    output out;
    input D0, D1, D2, D3, D4, D5, D6, D7, S0, S1, S2;
    wire Sobar, S1bar, T1, T2, T3, T4, T5, T6, T7, T8;
    not_gate U1 (S1bar, S1);
    not_gate U2 (S0bar, S0);
    not_gate U3 (S2bar, S2);
    and_gate U4 (T1, D0, Sobar, S1bar, S2bar);
    and_gate U5 (T2, D1, S0, S1bar, S2bar);
    and_gate U6 (T3, D2, Sobar, S1, S2bar);
    and_gate U7 (T4, D3, S0, S1, S2bar);
    and_gate U8 (T5, D4, Sobar, S1bar, S2);
    and_gate U9 (T6, D5, S0, S1bar, S2);
    and_gate U10 (T7, D6, Sobar, S1, S2);
    and_gate U11 (T8, D7, S0, S1, S2);
    or_gate U12 (out, T1, T2, T3, T4, T5, T6, T7, T8);
endmodule

```

Test bench code

```

module top;
    wire out;
    reg D0, D1, D2, D3, D4, D5, D6, D7, S0, S1, S2;
    m81 m81_name (.D0(D0), .D1(D1), .D2(D2), .D3(D3), .D4(D4),
    .D5(D5), .D6(D6), .D7(D7), .S0(S0), .S1(S1), .S2(S2), .out(
    out));
    initial begin
        D0 = 1'b0; D1 = 1'b0; D2 = 1'b0; D3 = 1'b0; D4 = 1'b0; D5 = 1'b0;
        D6 = 1'b0; D7 = 1'b0; S0 = 1'b0; S1 = 1'b0; S2 = 1'b0;
    end

```

```
#500 $finish;
```

```
end
```

```
always # 1 D0 = ~ D0;
```

```
always # 2 D1 = ~ D1;
```

```
always # 3 D2 = ~ D2;
```

```
always # 4 D3 = ~ D3;
```

```
always # 5 D4 = ~ D4;
```

```
always # 6 D5 = ~ D5;
```

```
always # 7 D6 = ~ D6;
```

```
always # 8 D7 = ~ D7;
```

```
always # 9 S0 = ~ S0;
```

```
always # 10 S1 = ~ S1;
```

```
always # 11 S2 = ~ S2;
```

```
always @(D0 or D1 or D2 or D3 or D4 or D5 or D6 or D7 or  
S0 or S1 or S2)
```

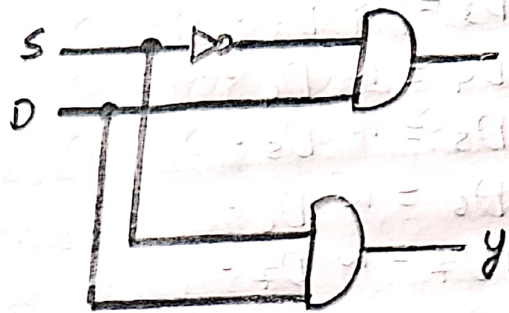
```
$monitor("At time = %t, output = %d", $time,  
out);
```

```
endmodule
```

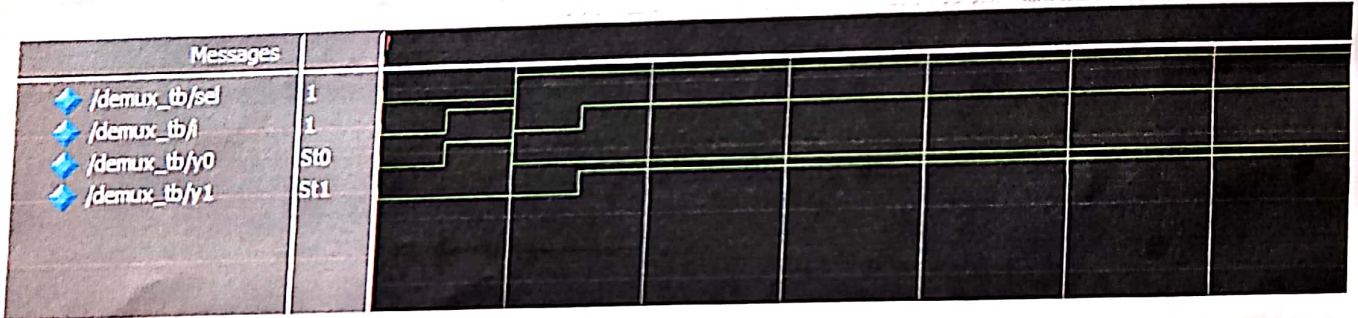

Truth table

S	D	y ₀	y ₁
0	0	0	0
0	1	0	1
1	0	0	0
1	1	0	0

Circuit diagram



Output



2:1 demultiplexer

Design verilog programs to implement types of De-Multiplexer.

1:2 Demultiplexer

```
module demux_1_2C
```

```
input sel;
```

```
input i;
```

```
output y0, y1;
```

```
);
```

```
assign {y0, y1} = sel ? {1'b0, i} : {i, 1'b0};
```

```
endmodule
```

Test bench code

```
module demux_tb;
```

```
reg sel, i;
```

```
wire y0, y1;
```

```
demux_1_2C demux(sel, i, y0, y1);
```

```
initial begin
```

```
$monitor("sel = %h: i = %h --> y0 = %h, y1 = %h",  
sel, i, y0, y1);
```

```
sel = 0, i = 0; #1;
```

```
sel = 0, i = 1; #1;
```

```
sel = 1, i = 0; #1;
```

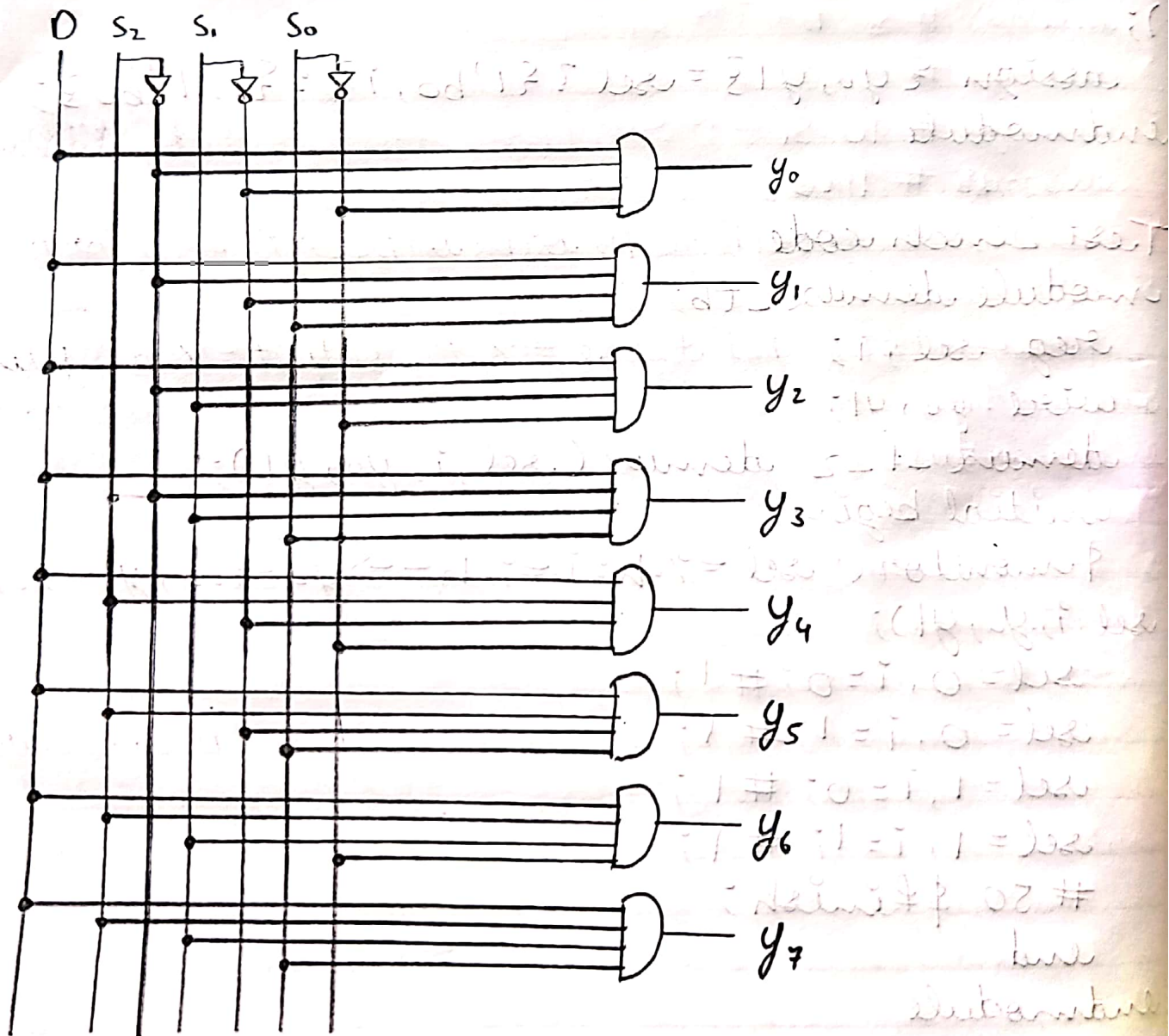
```
sel = 1, i = 1; #1;
```

```
#50 $finish;
```

```
end
```

```
endmodule
```


s_2	s_1	s_0	y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



1:8 Demultiplexer

```

module demux 1_8 (
    input D, input [2:0] S,
    output y0, y1, y2, y3, y4, y5, y6, y7);
    wire inv0, inv1, inv2;
    not not0 (inv0, S[0]);
    not not1 (inv1, S[1]);
    not not2 (inv2, S[2]);
    and and0 (y0, inv2, inv1, inv0, D);
    and and1 (y1, inv2, inv1, S[0], D);
    and and2 (y2, inv2, S[1], inv0, D);
    and and3 (y3, inv2, S[1], S[0], D);
    and and4 (y4, S[2], inv1, inv0, D);
    and and5 (y5, S[2], inv1, S[0], D);
    and and6 (y6, S[2], S[1], inv0, D);
    and and7 (y7, S[2], S[1], S[0], D);
endmodule
endmodule

```

Test bench

```

module tb;
    reg D;
    reg [2:0] S;
    wire y0, y1, y2, y3, y4, y5, y6, y7;
    demux (.D(D), .S(S), .y0(y0), .y1(y1), .y2(y2), .y3(y3),
    .y4(y4), .y5(y5), .y6(y6), .y7(y7));
    initial begin

```


Output:-

Messages									
◆ /b/D	0								
◆ /b/S	111	000	001	010	011	100	101	110	111
◆ /b/Y0	S0								
◆ /b/Y1	S0								
◆ /b/Y2	S0								
◆ /b/Y3	S0								
◆ /b/Y4	S0								
◆ /b/Y5	S0								
◆ /b/Y6	S0								
◆ /b/Y7	S0								

8:1 demultiplexer

DATE

EXPT. TITLE :

EXP. NO. 07

PAGE NO. 22

```
D=$random;
```

```
S=000
```

```
#10 S=001;
```

```
D=$random;
```

```
#10 S=010;
```

```
D=$random;
```

```
#10 S=011;
```

```
D=$random;
```

```
#10 S=100;
```

```
D=$random;
```

```
#10 S=101;
```

```
D=$random;
```

```
#10 S=110;
```

```
D=$random;
```

```
#10 S=111;
```

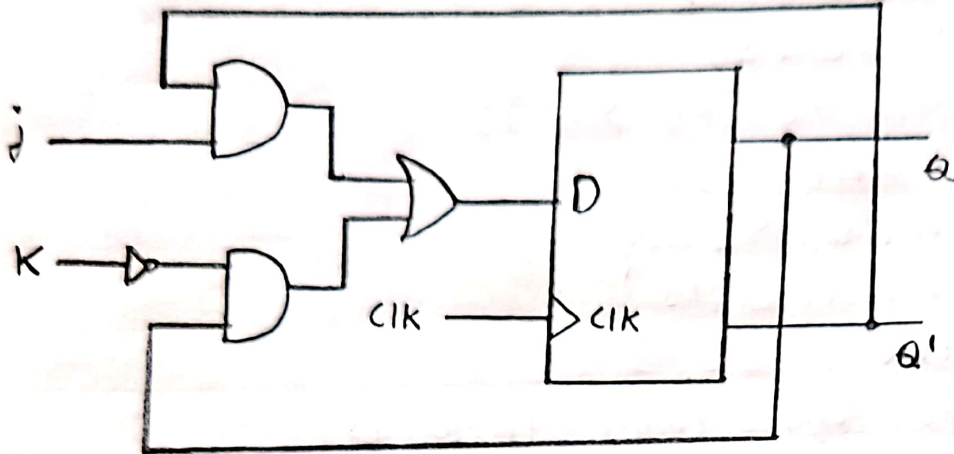
```
D=$random;
```

```
#10 $stop;
```

```
end
```

```
endmodule
```

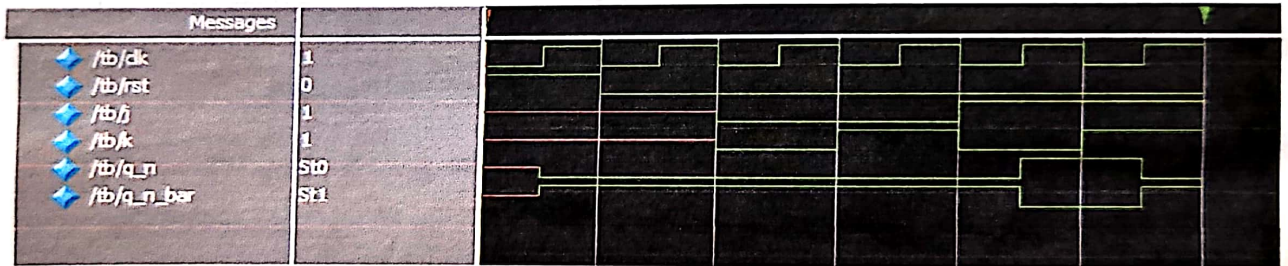

Circuit diagram :-



Truth Table :-

J	K	Q
0	0	Q
0	1	0
1	0	1
1	1	Q'

Output :-



JK Flip-Flop

Design verilog program for implementing various types of flip-flops such as SR, JK and D.

JK flip flops

```

module jk_ff(j, K, clk, reset, Q, Q_bar);
    input j, K, clk, reset;
    output reg Q, Q_bar;
    always @(posedge clk)
    begin
        if (~reset)
            Q, Q_bar <= {1'b0, 1'b1};
        else
            begin
                case ({j, K})
                    2'b00: Q, Q_bar <= {Q, Q_bar};
                    2'b01: Q, Q_bar <= {1'b0, 1'b1};
                    2'b10: Q, Q_bar <= {1'b1, 1'b0};
                    2'b11: Q, Q_bar <= {~Q, Q};
                    default: begin end
                end case
            end
        end
    end
endmodule

```

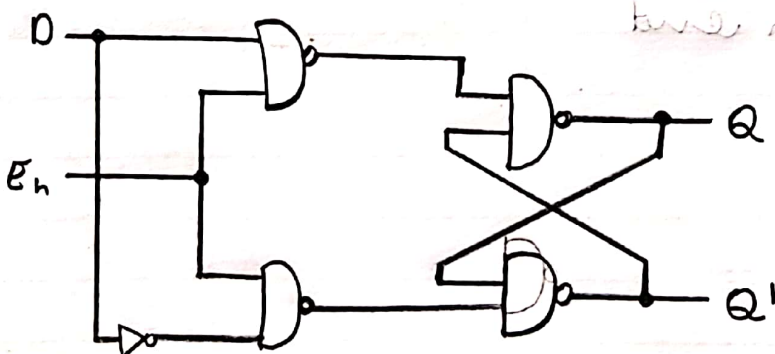
Test bench code

```

module tb();
    reg clk, rst, j, K;

```


Circuit diagram 3- Truth table 3-



E_n	D	Next state of Q
0	x	No change
1	0	0
1	1	1

above named test
test Duben
in the test results

we are given, q, n, q, n bar;

JK FF test design(j, K, CLK, rst, q, n, q, n bar);

initial begin

CLK = 0;

forever #5 CLK = ~CLK;

end

initial begin

rst = 1; #10;

rst = 0; #10;

j = 1'b0; K = 1'b0; #10;

j = 1'b0; K = 1'b1; #10;

j = 1'b1; K = 1'b0; #10;

j = 1'b1; K = 1'b1; #10;

\$finish;

end

endmodule

D - flip flop

module d_ff(Q, D, CLK, reset);

input D;

input CLK;

input reset;

output reg Q;

always @(posedge CLK or posedge reset)

begin

if(reset == 1'b1)

Q <= 1'b0;

250

روز شنبه ۱۳۰۲/۱۲/۱۲

1909

Output:

```
else
```

```
Q <= D;
```

```
end
```

```
endmodule
```

Test bench code

```
module tb;
```

```
reg D;
```

```
reg CLK; reg reset;
```

```
wire Q;
```

```
u1 uut(Q, D, CLK, reset);
```

```
initial begin
```

```
CLK = 1'b0;
```

```
forever #20 CLK = ~CLK;
```

```
end
```

```
initial begin
```

```
reset = 1'b1; #40;
```

```
reset = 1'b0; #40;
```

```
D = 1'b0; #40;
```

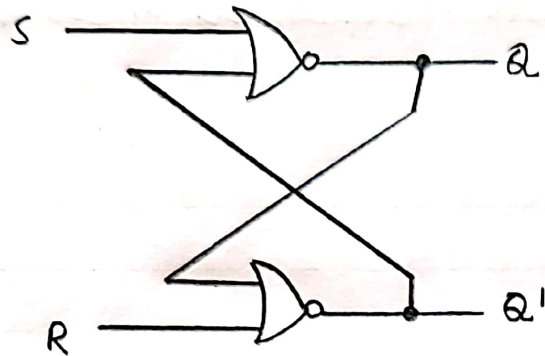
```
D = 1'b1; #40;
```

```
$finish;
```

```
end
```

```
endmodule
```


Circuit diagram:-

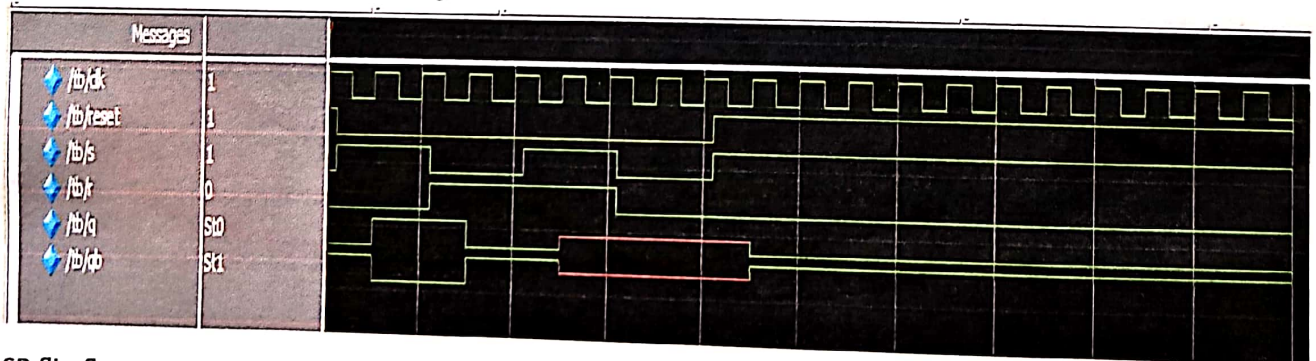


Truth table:-

S	R	Q	Q'
1	0	1	0
0	0	1	0
0	1	0	1
0	0	0	1
1	1	0	0

Output:-

change



SR flip-flop

SR - flip flop.

```
module srff(s,r,clk,q,qb);  
  input s,r,clk;  
  inout q,qb;  
  wire s1,r1;  
  assign s1 = ~(s & clk);  
  assign r1 = ~(r & clk);  
  assign q = ~(s1 & qb);  
  assign qb = ~(r1 & q);  
endmodule
```

Test bench code

```
module srff_tb;  
  reg s,r,clk;  
  wire q,qb;  
  srff srff_tb(s,r,clk,q,qb);  
  initial begin  
    forever  
      begin  
        clk = 0;  
        #50 clk = 1;  
        #50 clk = 0;  
      end  
    end  
  initial begin  
    s = 0; r = 1;  
    #10 s = 0; r = 0;
```


DATE

EXPT. TITLE :

EXP. NO. 02

PAGE NO. 27

#10 S=1; R=0;

#10 S=1; R=1;

end

endmodule